WUNDERDOG

# **Jakamo x Wunderdog –** AI-native development

One-day training

# Jakamo x Wunderdog – AI-native development

Opening slide. Say who you are and what today is about: how to work with coding agents so that the team actually gets faster, not just the typing.

The whole day builds toward one idea: humans stop reading every line of code. Machines check that the code is correct. Humans check that it is the right thing to build.

Key points
- One day, five parts, then an optional hackathon
- Goal: code review that scales when agents write the code

# Trainer



## Mikko Vallaskivi

**Technology Lead**

- Software development generalist
- 10+ years of full stack web, mobile and cloud experience
- 15+ years of programming
- Copilot/Codex/Claude Code

# Trainer

Introduce yourself. Keep it short: who you are, what you do at Wunderdog, and why you are the one running this day.

Mention the project you will talk about at the end of part one (about 6,000 developers, about 300 teams), so the room knows the advice comes from real work.

Key points

- Mikko Vallaskivi, Technology Lead at Wunderdog
- Software development generalist, 10+ years full stack, 15+ years programming
- Uses Copilot, Codex and Claude Code

WUNDERDOG

01

# Where AI is today

# 01 Where AI is today

Part one: what the data says about AI and real productivity. Most numbers come from DX's talk "The State of AI in Software Development" (Justin Reock, AI Engineer, 2026), based on data from about 200,000 engineers in 400+ companies.

Check before the talk: the numbers in this part were taken from summaries of the talk. Watch the video once and confirm them.

Sources
https://www.youtube.com/watch?v=Se8jHLIiLXE

# How much faster does AI make a dev team?

Vote with your fingers. One finger for 10%, four fingers for 10x.

| 1 finger | 2 fingers | 3 fingers | 4 fingers |
|---|---|---|---|
| 10% | 50% | 2x | 10x |

Remember the most common answer. We come back to it.

# How much faster does AI make a team?

Ask the room to vote with their fingers: 1 = 10% faster, 2 = 50% faster, 3 = 2x, 4 = 10x. Look around and say the most common answer out loud. Write it down, because slide "The answer" compares it with the data.

Key points

- Everyone votes, no wrong answers
- Remember the room's answer

# The hype

“10x engineers”

“AI writes half of our code”

“Build a product in a weekend”

What happens when you measure it?

# The hype

These are the kinds of headlines everyone has seen. They are not quotes from one person; they are the general message in the news and on social media.

The question for the next slides: what happens when someone actually measures it across many companies?

Key points
• Headlines promise huge gains
• Let's look at measured data

# The data

**200,000**
engineers

**400+**
companies

DX, “The State of AI in Software Development”, AI Engineer 2026

# The data

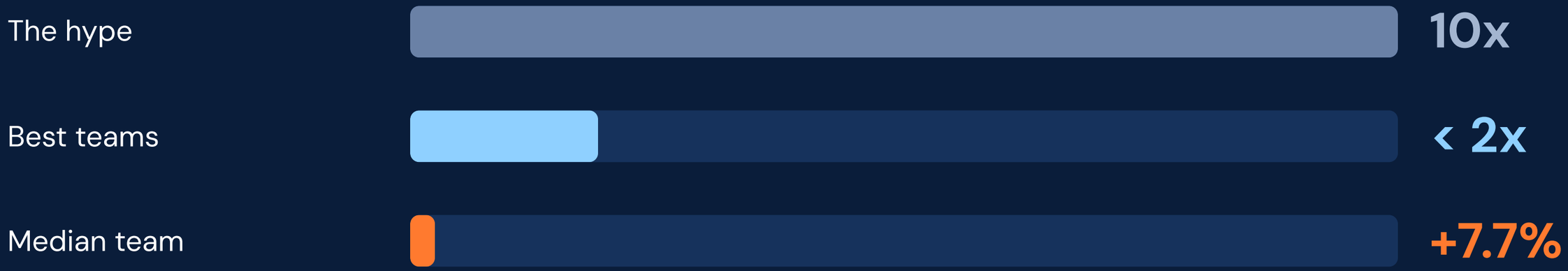
DX measures developer productivity in many companies. In this talk Justin Reock (Deputy CTO at DX) shows their data on about 200,000 engineers in more than 400 companies. This is not a survey of opinions; it combines system data (PRs, deploys) and developer feedback.

Key points
- ~200,000 engineers
- 400+ companies
- Measured, not guessed

Sources
https://www.youtube.com/watch?v=Se8jHLIiLXE

# The answer

Change in PR throughput after AI tools




Compare that with your vote.

# The answer

This is the "wait, what?" moment. The median gain in PR throughput is about 7.7%. Even the best teams in the data did not reach 2x. Compare with what the room voted.

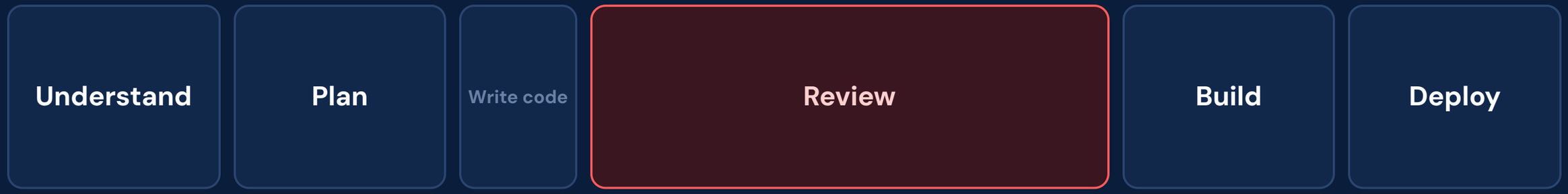
The bars are on a simple scale: 10x = full bar. 7.7% faster is a tiny bar next to it.

Key points
• Median: about 7.7% more PR throughput
• Even top teams: under 2x
• Far from the hype

Check before the talk: confirm 7.7% and "under 2x" in the video.

Sources
https://www.youtube.com/watch?v=Se8jHLIiLXE
https://daily.dev/posts/the-state-of-ai-in-software-development-data-from-400-orgs-justin-reock-dx-9leiovhx3

# Where the time goes




- AI made writing code fast
- The rest of the work stayed the same. Review got bigger.
- Writing code was never the slow part

# Where the time goes

The reason the gains are small: writing code was never the bottleneck. The slow parts are understanding legacy code, coordination, code review, builds and handling incidents.

Watch the animation: "Write code" shrinks because AI made it fast. "Review" grows and turns red, because more code now waits for humans to read it.

Key points
- Code generation got faster
- Review, builds and coordination did not
- The bottleneck moved to review

Sources
https://www.youtube.com/watch?v=Se8jHLliLXE

# Review is drowning

PR SIZE

**72**

lines per PR, up from 44. Bigger PRs are harder to review.

DEPLOYS

## More often, less stable

Deployment frequency goes up. The change failure rate swings much more.

CONFIDENCE

## Down

Maintainability scores go up, but developers feel less sure their changes are safe.

# Review is drowning

Three side effects from the DX data.

1. PRs grew from about 44 to 72 lines. Fast code generation plus slow builds pushes people to batch more into one PR. Bigger PRs take longer to review and hide more problems.
2. Teams deploy more often, but the change failure rate is much less stable than before.
3. Maintainability looks better on paper, but developers are less confident that their changes are safe.

All three put more pressure on review.

Check before the talk: confirm 44 → 72 lines in the video.

Sources
https://www.youtube.com/watch?v=Se8jHLliLXE

# Using more AI ≠ saving more time




## Why do you think that is?

# Using more AI is not saving more time

Juniors use AI the most. Staff+ engineers use less, spend fewer tokens, and save about the same amount of time.

The bars show the pattern, not exact numbers.

Ask the room (1 minute): why? Typical answers: seniors know what to ask for, they give better context, they know when to stop, they review the output better. That is exactly what the rest of the day is about.

Key points
- More usage does not mean more value
- How you use it matters more than how much

Sources
https://www.youtube.com/watch?v=Se8jHLliLXE

# Good DX = good agent experience

Fast builds | Good tests | Clear rules | Written decisions — help people and agents alike

Morgan Stanley

**~300,000 hours saved per year**

Zapier

**15% more value per engineer, and still hiring**

Spotify

**An agent that helps SREs with incidents**

# Good DX is good agent experience

The companies that get real gains improved the whole system, not just the code generation. A codebase that is easy for people to work in (fast builds, good tests, clear rules, written decisions) is also easy for agents. You can even ask the agents where they get stuck.

Examples from the DX talk:
• Morgan Stanley: about 300,000 hours saved per year
• Zapier: about 15% more value per engineer, and they are hiring more
• Spotify: an agent that helps SREs during incidents

Check before the talk: confirm the company numbers in the video.

Sources
https://www.youtube.com/watch?v=Se8jHLliLXE

# Today's goal

## Machines check the code

Is the code what it should be?



## Humans check the intent

Is this the right thing to build?

Review stops being the bottleneck.

# Today's goal

This is the goal for the whole day. Review is split into two layers.

Machines check the code: types, lint, tests, static analysis, evals. Is the code what it is supposed to be?
Humans check the intent: is this the right thing to build, and does it match what we planned?

Every part of today moves work from human review to machine review, so humans only review the intent.

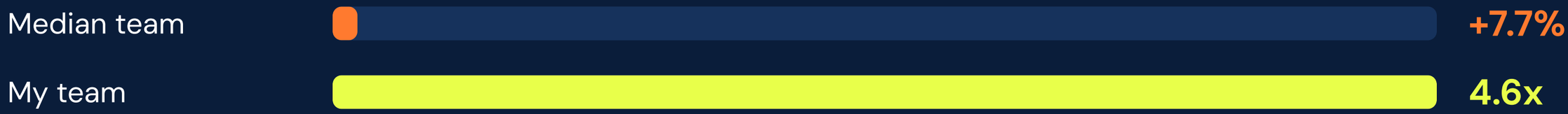
Key points
- Two review layers
- Machines: correctness
- Humans: intent

# My project

**~6,000**
developers

**~300**
teams

**4.6x**
more output in my team, in an ultra-regulated environment




Today I'll show you how.

# My project

Your own story. The project has about 6,000 developers in about 300 teams. Your team got 4.6x more output, in an ultra-regulated environment.

Put it next to the 7.7% median: the difference is not the model. It is the setup, the planning and the review process. That is what today is about.

Be ready for the question "what does output mean?" Say how it was measured (PRs merged, features shipped or similar) and over what time.

Key points
- ~6,000 developers, ~300 teams
- My team: 4.6x more output
- Same models everyone has, different way of working

WUNDERDOG

02

# Agent setup

# 02 Agent setup

Part two: how to set up the agent so we can trust the code it writes. We go from instructions Claude may follow to checks Claude Code always runs. Then MCP, where the agent files live, and evals to prove the setup works.

All examples are for Claude Code on Jakamo's stack: .NET, PHP, Next.js/React and Azure.

Sources
https://code.claude.com/docs/en/best-practices

# Same agent, different results

SAME MODEL · SAME PROMPT Add a phone number field to the user profile

TEAM A

- ✗ New validation helper, copied twice
- ✗ No tests
- ✗ Edited the migration by hand

TEAM B

- ✓ Used the existing validator
- ✓ Test for each case
- ✓ Migration made with the team's skill

The difference is the setup.

# Same agent, different results

Two teams, the same model, the same prompt. One gets messy code, the other gets code that fits the codebase. The model did not change. What changed is what the agent knew and what it was allowed to do.

This part is about that setup.

Key points
- Same model, same prompt
- Different setup → different code

# From “please” to “must”

CLAUDE DECIDES

CLAUDE CODE ENFORCES

| | |
|---|---|
| **Rules** | CLAUDE.md, loaded in every session |
| **Specific rules** | Only for some files |
| **Skills** | Step-by-step procedures |
| **Subagents** | Custom helpers with their own tools |
| **Hooks** | Run every time, no matter what |

# From please to must

Every instruction you give Claude Code is either a request or a guarantee.

At the top, Claude (the model) reads the instruction and decides what to do. It usually follows it, but there is no guarantee.
At the bottom, Claude Code (the program around the model) runs the check every time, no matter what the model decided.

We go down this list one step at a time. The further down, the more certain.

Key points
• Top: requests, the model decides
• Bottom: guarantees, the harness enforces
• Same idea as the two review layers

Sources
https://code.claude.com/docs/en/best-practices

# Rules

- Loads in every session
- Only what Claude can't see in the code: commands, misleading places, team decisions, anything you corrected twice
- Under 200 lines. `/init` to start, `/doctor prompt-audit` to clean
- Claude may still skip a line

CLAUDE.md

```
# Commands
- Test: dotnet test
- One PHP file: vendor/bin/phpstan analyse src/X.php
- Web types: pnpm tsc --noEmit

# Conventions
- Use Result<T> from Core/Errors, never throw
- legacy/ is live, don't copy its patterns

# Gotchas
- Integration tests need the Azurite
  emulator running
```

# Rules (CLAUDE.md)

CLAUDE.md is loaded into every session, so every line costs a bit of attention on every task. Keep only what Claude can't work out from the code:

- the exact test, typecheck and lint commands
- places where the code is misleading (old patterns that must not be copied)
- decisions that are not in the repo
- anything you have had to correct twice

Stay under about 200 lines. /init writes a first version from the codebase. /doctor prompt-audit finds instructions that are outdated, point at missing files or contradict each other.

This is still a request. Claude follows it nearly every time, but not always.

The example file is made up for the stack; replace it with a real Jakamo one if you can.

Sources

https://code.claude.com/docs/en/best-practices

https://code.claude.com/docs/en/memory

# Specific rules

```
repo/
├─ src/Api/          .NET
├─ backend/          PHP
├─ web/              Next.js
└─ .claude/rules/
   ├─ api.md
   ├─ backend.md
   └─ web.md
```

```
.claude/rules/api.md

---
paths:
  - "src/Api/**/*.cs"
---
- Validate requests with FluentValidation
- Every endpoint needs an integration test
```

- Loads only when Claude works in those files
- Less noise, better focus. Still a request.

# Specific rules

If a rule only matters in one part of the code, put it in .claude/rules/ with a paths: filter. It loads only when Claude works on matching files. A .NET rule for the API, a PHP rule for the backend, a React rule for the web app.

This keeps CLAUDE.md short and keeps unrelated rules out of the way. It is still a request: Claude decides.

The rule text in the example is made up; use Jakamo's real conventions.

Sources
https://code.claude.com/docs/en/memory
https://code.claude.com/docs/en/best-practices

# Skills

AT START: NAME + DESCRIPTION

```
name: add-ef-migration
description: Add an EF Core migration
  for a model change, and test it.
```

→

WHEN NEEDED: THE FULL STEPS

1. Change the model in Domain/
2. dotnet ef migrations add <Name>
3. Read the generated file, no data loss
4. Run the migration test
5. Report the command output

- Claude picks it up itself, or you start it with `/name`

# Skills

A skill is a step-by-step procedure for one job, saved in .claude/skills/<name>/SKILL.md.

Only the name and description load at the start of a session. The full steps load only when the task needs them. That makes skills lighter than putting the same text in CLAUDE.md.

Claude can start a skill by itself when the description matches the task, or you can start it with /name.

Key points
- A procedure for one job
- Only the description loads up front
- Claude picks it up, or you call it

Sources
https://code.claude.com/docs/en/skills
https://code.claude.com/docs/en/best-practices

# Skill examples

`/add-ef-migration`

## Add an EF Core migration

Claude can start it

`/run-local`

## Run the app locally

API, PHP app and web app, with the right env

`/deploy-staging`

## Deploy to Azure staging

`disable-model-invocation: true` → only a human can start it

TEST THE DESCRIPTION

✓ “add a column to Orders” → loads

✗ “review this PR” → does not load

# Skill examples

Three examples for Jakamo:
• Add an EF Core migration: Claude can start it on its own.
• Run the app locally: so /run and /verify don't have to work out the steps every time.
• Deploy to Azure staging: it has side effects, so it gets disable-model-invocation: true. Claude can't start it; only a human typing /deploy-staging can.

Test the description too. Claude decides from the description whether to load the skill. Write prompts that should load it and near misses that should not.

Sources
https://code.claude.com/docs/en/skills
https://agentskills.io/skill-creation/optimizing-descriptions

# Subagents

**.NET API security reviewer**

**Migration checker**

**Test writer**

- Own context, own tools, one use case
- Not smarter. Focused and fresh. Costs extra tokens.

```
.claude/agents/security-reviewer.md

---
name: security-reviewer
description: Reviews .NET API changes
  for auth and input problems.
tools: Read, Grep, Glob
---
You review code you didn't write.
Report only real security problems.
Skip style.
```

tools: Read, Grep, Glob = it can't edit. Enforced.

# Subagents

A subagent is a custom helper for one use case, saved in .claude/agents/. It has its own context and its own list of tools.

Good uses: work that can run in parallel, keeping long output out of your main session, a fresh reviewer that didn't see how the code was written, and giving a task fewer tools.

The tools line is enforced by Claude Code: a reviewer with only Read, Grep and Glob can't edit files, no matter what the model wants. That is why subagents sit low on the ladder.

They are not smarter: same model, same task. Each one costs extra tokens.

Sources
https://code.claude.com/docs/en/sub-agents
https://code.claude.com/docs/en/best-practices

# Hooks

PostToolUse

## After every edit

`dotnet format` · `php-cs-fixer` · `prettier`

PreToolUse

## Before an edit

Block edits to `Migrations/` and generated API clients

Stop

## Before Claude finishes

Tests must pass. Exit code 2 = keep working.

```
// .claude/settings.json
"PreToolUse": [{ "matcher": "Edit|Write",
  "hooks": [{ "type": "command", "command": "./scripts/block-migrations.sh" }] }]
```

# Hooks

A hook runs your own command at a fixed point in Claude's loop. It runs every time, no matter what the model decides. This is the first real guarantee.

Three examples:
• PostToolUse: after every Edit or Write, run the formatter (dotnet format, php-cs-fixer, prettier).
• PreToolUse: before an edit, block changes to the Migrations folder or to generated API clients. Exit code 2 blocks the action.
• Stop: when Claude tries to finish, run the tests. If they fail, exit code 2 sends Claude back to work.

Live demo idea: ask Claude to edit a migration. The hook blocks it and Claude explains why.

Sources
https://code.claude.com/docs/en/hooks-guide
https://code.claude.com/docs/en/best-practices

# Keep guarantees few

STAYS A RULE

“Prefer named exports”

Has exceptions. Let Claude judge.

BECOMES A HOOK

“Never edit Migrations/ by hand”

Zero exceptions. Enforce it.

More certain, but more to maintain. 40 hooks = mostly preferences.

# Keep guarantees few

The further down the ladder, the more certain, but also the more you have to maintain. Every hook will sometimes fire when it shouldn't.

Use hooks only for rules with zero exceptions. Preferences stay in rules, where Claude can judge whether they fit the situation.

If you keep adding IMPORTANT to a CLAUDE.md line and Claude still misses it, that line belongs in a hook. If you have hooks for preferences, delete them.

Sources
https://code.claude.com/docs/en/best-practices

# MCP: why it's useful

**Azure DevOps**
Boards, work items

**Database**
Query real schemas

**Browser**
Click through the app

**Error tracking**
Read the real errors

- Connects the agent to systems outside the repo
- Typed tools, sign-in with OAuth
- Best for systems with no CLI, and tools you use often

# MCP: why it's useful

MCP (Model Context Protocol) connects the agent to systems outside the repo: work items, databases, a browser, error tracking. The agent gets typed tools and can sign in with OAuth.

It is most useful for systems that have no command-line tool, and for tools you use often in the same session.

Key points
- Connects to outside systems
- Typed tools, OAuth
- Best when there is no CLI

Sources
https://code.claude.com/docs/en/mcp

# MCP: the cons

CONTEXT COST

## 550–1,400 tokens per tool

Loaded before any work. 3 servers ≈ 55,000 tokens.

RELIABILITY

## Remote servers time out

One benchmark saw 28% failures on a remote server.

SECURITY

## Prompt injection

Servers that read outside content can bring in hidden instructions.

TRUST

## Approve each server

Turn off the ones you don't use.

# MCP: the cons

The downsides of MCP:
• Context cost: tool definitions are loaded before any work starts. One article measured 550–1,400 tokens per tool, and about 55,000 tokens for three servers (GitHub, Slack, Sentry).
• Reliability: remote servers can fail and time out. One benchmark saw a 28% failure rate on a remote server.
• Security: servers that read outside content (web pages, issues, emails) can bring in prompt injection.
• Trust: check each server before connecting it, and turn off the ones you don't use.

Fair note: Claude Code's tool search is on by default, so it loads tool names first and full definitions when needed. That lowers the up-front cost. The numbers above come from a vendor article that prefers CLIs; present them as an example, not a law.

Sources
https://www.apideck.com/blog/mcp-server-eating-context-window-cli-alternative
https://code.claude.com/docs/en/mcp

# MCP vs CLI: tokens

Same task: check which languages a repo uses




CLI first: `az`, `gh`, `dotnet`. MCP when there is no CLI.

Scalekit benchmark via Apideck. Benchmarks show 4–32x more tokens with MCP.

# MCP vs CLI: tokens

The same small task done two ways: with the GitHub MCP server it used 44,026 tokens (43 tool definitions loaded). With the CLI it used 1,365 tokens. Benchmarks in the same article show MCP using 4 to 32 times more tokens.

Rule of thumb: if a system has a CLI (az, gh, dotnet), let Claude use it. It already knows most CLIs, and for a new one, pointing it at --help is enough. Use MCP when there is no CLI.

Fair note: tool search in Claude Code lowers MCP's up-front cost. The numbers come from a vendor that prefers CLIs.

Sources
https://www.apideck.com/blog/mcp-server-eating-context-window-cli-alternative
https://code.claude.com/docs/en/best-practices

# Where the files live

LOCAL · ONLY YOU

## Your laptop

`~/.claude/`
`CLAUDE.local.md`
`settings.local.json`

Personal preferences. A teammate never sees them.

REPO · THE TEAM

## The repo

`CLAUDE.md`
`.claude/rules · skills · agents`
`.claude/settings.json`
`.mcp.json`

Committed and reviewed in PRs, like code.

CENTRAL · THE COMPANY

## One place

`Managed settings`
`Company plugin marketplace`
`Org connectors`

One setup for 300 teams. Managed settings can't be overridden.

# Where the files live

Where a file lives decides who gets it.

Local: ~/.claude/, CLAUDE.local.md and .claude/settings.local.json are only yours. Good for personal preferences. When you click "Yes, and don't ask again", the rule goes into the local file.

Repo: CLAUDE.md, .claude/rules, skills, agents, settings.json and .mcp.json are committed. Everyone working in the repo gets them, and changes are reviewed in PRs like code.

Central: managed settings (from the admin console or MDM), a company plugin marketplace and org connectors. One place for many teams. No personal settings file can override managed settings, so this is the place for security policy.

Sources
https://code.claude.com/docs/en/best-practices
https://code.claude.com/docs/en/settings

# Local: only you

```
~/.claude/
├─ CLAUDE.md          all your projects
├─ settings.json
└─ skills/
repo/
├─ CLAUDE.local.md    gitignored
└─ .claude/settings.local.json
```

- Your style, your shortcuts, your allowed commands
- A teammate never gets it
- If the team needs it, it doesn't belong here

# Local

Local files are for you only: your personal CLAUDE.md in ~/.claude/, your own skills, CLAUDE.local.md in the repo (add it to .gitignore) and .claude/settings.local.json.

The trap: something works great for you because of your local setup, and a teammate gets a different result. If the team needs a rule, move it to the repo.

Sources
https://code.claude.com/docs/en/memory
https://code.claude.com/docs/en/settings

# Repo: the team

```
repo/
├─ CLAUDE.md
├─ .mcp.json
└─ .claude/
   ├─ settings.json   hooks, permissions
   ├─ rules/
   ├─ skills/
   └─ agents/
```

- Committed, so everyone in the repo gets it
- Reviewed in PRs like code
- Hooks and allow rules start after each person trusts the folder

# Repo

Repo files are committed, so everyone working in the repo gets the same setup. Changes to them go through PRs like any code change, which is how the team agrees on rules.

Detail worth knowing: deny rules in .claude/settings.json apply right away. Allow rules and hooks start after each person accepts the trust dialog for that folder.

Sources
https://code.claude.com/docs/en/best-practices
https://code.claude.com/docs/en/permissions

# Central: the company

### Managed settings

From the admin console or MDM. Nobody can override them.

### Plugin marketplace

Shared skills, agents and hooks, installed by every team.

### Org connectors

MCP servers the company has approved.

One setup for 300 teams. The place for security policy.

# Centralized

Central files belong to the organization.

• Managed settings come from the admin console, MDM or a managed-settings.json file. No personal settings can override them, so security rules go here.

• A company plugin marketplace shares skills, agents and hooks across many repos and teams.

• Org connectors are MCP servers the company has approved.

With 300 teams, this is how one good practice reaches everyone without copying files.

Sources

https://code.claude.com/docs/en/managed-settings

https://code.claude.com/docs/en/plugins

# Your setup is code too: test it

PROMPT Add an EF Core migration for the phone field




One good run proves nothing.

# Your setup is code too: test it

Rules, skills and hooks are code too, and they change. How do you know a change made things better?

Agents are not deterministic. The same prompt can give different results. If you try once and it looks fine, you have not proven anything. That is why evals run the same task several times and count passes.

Sources
https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

# What an eval is



## Task

Input and what counts as success



## Trials

Several runs of the same task

tests pass

skill used

rubric

## Graders

Each checks one thing

67%

## Score

Pass rate across trials

# What an eval is

The words we use, from Anthropic's eval post:

- Task: one test with an input and what counts as success.
- Trials: several runs of the same task.
- Graders: each checks one thing (tests pass, a skill was used, a rubric).
- Score: the pass rate across trials. 67% = 2 of 3.

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

# Grade the outcome, not the claim

WHAT THE AGENT SAYS

> Fixed the flaky test. CI is green.

WHAT THE REPO SAYS

```
$ dotnet test --filter Auth (x10)
7 runs passed
3 runs failed: token expiry
```

CLAIM ≠ OUTCOME

- Check the end state
- Code graders first: fast, cheap, same answer every time

# Grade the outcome, not the claim

Agents say "done" confidently even when it isn't. Graders should check the real result in the repo (tests, files, data), not the agent's message.

Three kinds of graders: code graders (tests, regex, tool checks) are fast, cheap and give the same answer every time. Model graders (an LLM with a rubric) handle open-ended output but need checking. Humans are best but slow and expensive. Use code graders where you can.

The test output is example data.

Sources
https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

# Claude builds the evals for you

```
$ claude plugin eval init
  asks about your skill, drafts cases and graders, tries them once
$ claude plugin eval .
```

| case | with | without | Δ |
|---|---|---|---|
| uses-migration-skill | 1.00 | 0.33 | +0.67 |
| no-data-loss | 1.00 | 0.67 | +0.33 |
| runs-migration-test | 0.67 | 0.67 | 0.00 |

3 runs per case, with and without the skill · report.html in `evals/results/` · `--threshold` fails CI

# Claude builds the evals for you

Claude Code has evals built in, for skills and plugins.

- claude plugin eval init asks about your skill, drafts test cases and graders, tries them once and writes the files.
- claude plugin eval . runs every case 3 times with the skill and 3 times without it, and shows both scores and the difference.
- Results go to evals/results/ with a report.html.
- In CI, --threshold fails the build when a case drops below the limit.

Read the Δ column: 0.00 means the skill adds nothing on that case. The numbers in the table are example data.

Sources

https://code.claude.com/docs/en/plugin-evals

# Start small, from real failures

Bug tracker | Review comments | Failed agent runs | Manual checks you already do

**50**

20–50 tasks is enough

- Measure with and without the skill: the difference is what it adds
- Rerun on every setup change and every new model

# Start small, from real failures

Don't invent test cases. Take them from things that actually went wrong: the bug tracker, review comments, failed agent runs, and checks you already do by hand. Anthropic says 20 to 50 simple tasks from real failures is a great start.

Measure with and without the skill. Rerun the suite every time you change the setup and every time a new model comes out.

Sources
https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
https://code.claude.com/docs/en/plugin-evals

# Fix the eval, not just the agent

Same model, same benchmark: **42%** → **95%**

### Clear tasks
Two experts agree on pass or fail

### Fair graders
Not too strict

### Clean runs
No leftovers between trials

### Read transcripts
They show why it failed

A tested setup is what lets machines review the code.

# Fix the eval, not just the agent

Anthropic's post has a good example: Opus 4.5 first scored 42% on CORE-Bench. Someone found rigid grading, unclear tasks and tasks that could not be reproduced. After fixing the eval, the same model scored 95%. The model did not change; the eval did.

Four habits: clear tasks (two experts would agree on pass or fail), fair graders, a clean environment for every run, and reading the transcripts.

This closes part two: a tested setup is what lets us trust the code the agent writes.

Sources
https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

WUNDERDOG

# 03

# Talking to the agent

# 03 Talking to the agent

Part three is short: how you talk to the agent in a session. The setup from part two is the base; this is about the prompts you type every day.

Sources
https://code.claude.com/docs/en/best-practices

# Tell it what it can't see

VAGUE

```
fix the login bug
```

SPECIFIC

Login fails after the session times out. Look at token refresh in the .NET auth service. Write a failing test first, then fix it.

The file | The symptom | What “done” looks like

# Tell it what it can't see

The agent reads the repo, but it can't see what's in your head: the symptom you saw, which part of the code you suspect, or what "fixed" means.

A specific prompt names the file or area, describes the symptom, and says what done looks like (here: a failing test first, then the fix). It saves you a round of corrections.

A vague prompt is fine when you are exploring. When you know what you want, say it.

Sources
https://code.claude.com/docs/en/best-practices

# Hand it over, and say how big

`@file`

## Point to the file

`@src/Api/Auth/TokenService.cs`

`screenshot`

## Paste the broken page

The Next.js page as the user sees it

`pipe`

## Pipe in the log

```
cat azure.log | claude -p "why?"
```

SAY HOW BIG

Claude builds the smallest thing that works. Say “be ambitious” when you want edge cases, empty states and error states too.

# Hand it over, and say how big

If you already have what Claude needs, hand it over instead of describing it: @ a file to include it, paste a screenshot of the broken page, pipe a log into claude -p.

Claude tends to build the smallest thing that satisfies the request, because it can't see how much you hoped for. "Be ambitious" tells it you want the fuller version: edge cases, empty and error states, the follow-up work. That is about scope, which only you know.

Sources
https://code.claude.com/docs/en/best-practices

# Ask for evidence

```
> Run dotnet test and keep going until it passes. Show me the command and its output.
```

NOT ENOUGH

“All tests pass.”

EVIDENCE

```
$ dotnet test
Passed! Failed: 0, Passed: 214
```

Reading the output is faster than running it again yourself.

# Ask for evidence

Give Claude something it can run that comes back pass or fail, and ask it to keep going until it passes. Then ask for evidence: the command it ran and the output. "All tests pass" on its own tells you very little.

Reading the output is faster than running the check yourself. For a session you weren't watching, it's the only proof you have.

Sources
https://code.claude.com/docs/en/best-practices

# Steer the session

## Watch the first minute.

`Esc`

### Stop

Stops Claude and keeps the work so far. Redirect.

`/rewind`

### Go back

Roll back the code, the chat or both to an earlier prompt.

`/clear`

### Start over

Between tasks, and after two failed corrections.

# Steer the session

No setup replaces watching the first minute and stepping in when Claude heads the wrong way.

- Esc stops Claude mid-action and keeps everything so far.
- /rewind (or Esc twice) rolls back the code, the conversation or both. That makes it safe to try a bold approach first.
- /clear starts over. Use it between unrelated tasks. If you have corrected the same thing twice and it's still wrong, a third correction just adds to a chat full of failed attempts. /clear and one better prompt works better.

Sources

https://code.claude.com/docs/en/best-practices

https://code.claude.com/docs/en/checkpointing

# Call skills directly

```
> /implement JIRA-ISSUE-{YYYY}
> /design 3 user profile management section examples
```

### Short

One command replaces a long prompt

### Same steps every time

Read the ticket, plan, build, test, open the PR

### Easier to review

Same for juniors and seniors. Results look the same.

# Call skills directly

Skills can be called directly with arguments. Two examples:
• /implement JIRA-ISSUE-{YYYY}: reads the ticket, plans, builds, tests and opens the PR, the team's way.
• /design 3 user profile management section examples: gives three design options to choose from.

Why it's great: one short command replaces a long prompt. The team's way of working is built into the skill, so every run follows the same steps, for juniors and seniors alike. Results that look the same are easier to review. Remember the junior vs staff+ slide from part one.

Open question: do Jakamo teams already have skills like these, or do we show demo ones?

# Myth or still true?

Vote: thumbs up = still true, thumbs down = myth

| | |
|---|---|
| Write IMPORTANT in CAPS so Claude follows the rule | **Myth** |
| Say “think harder” for a better answer | **Myth** |
| Tell it “don't be lazy, keep going” | **Myth** |
| Describe your architecture in CLAUDE.md | **Myth** |

# Myth or still true?

Quick vote on each line, then reveal. All four are outdated advice:

- CAPS: current models follow a plainly written line. Too much emphasis makes Claude over-apply the rule. If a rule can't bend, make it a hook.
- "Think harder": effort level is the real control. These words only nudge within it.
- "Don't be lazy, keep going": current models carry on through an agreed task. Give it a check to pass instead.
- Architecture in CLAUDE.md: Claude reads the repo itself. Only write what isn't in there.

Sources

https://code.claude.com/docs/en/best-practices

WUNDERDOG

# 04

# Planning

# 04 Planning

Part four, after lunch: planning. After part two we can trust the code. What's left is the intent: are we building the right thing? Intent is what we plan for.

Running example for this part: adding a phone number to the user profile.

# We trust the code. Now what?




So we start with discovery, then plan. Intent is what we prove and validate.

# We trust the code. Now what?

Part two made the code checkable by machines. But nothing checks that we built the right thing. That is why we start with a discovery session and then move to planning. Intent is what we aim to prove and validate.

Key points
- Code: machines check it
- Intent: needs discovery and a plan

# A plan without ultra-precise acceptance criteria is not a plan.

## It's a road sign to oblivion.

The agent can't build what can't be verified.

# A road sign to oblivion

Big statement, say it slowly. A plan without ultra-precise acceptance criteria (ACs) is not a plan. The agent can't build what can't be verified.

The rest of this part shows how to get those ACs.

# Vague AC vs precise AC

VAGUE

“User can update their profile.”

PRECISE: ONE BEHAVIOR EACH

**AC1** Given a logged-in user, when they save a phone number that isn't in E.164 format, then the save is blocked and the field shows “Invalid phone number”.

**AC2** Given a logged-in user, when they save their profile more than 2 times within 1 second, then the third save is rejected with HTTP 429.

# Vague AC vs precise AC

Left: what most tickets say. Right: two precise ACs, each with one behavior.

AC1: Given a logged-in user, when they save a phone number that isn't in E.164 format, then the save is blocked and the field shows "Invalid phone number".
AC2: Given a logged-in user, when they save their profile more than 2 times within 1 second, then the third save is rejected with HTTP 429.

Each AC has concrete values and a result you can test.

# What makes an AC precise

1. One behavior per AC
2. Concrete values, not adjectives
3. Error cases and permissions included
4. Out of scope written down
5. Two people would agree on pass or fail

If you can't write the test, the AC isn't done.

# What makes an AC precise

Five checks for an AC:
1. One behavior per AC.
2. Concrete values, not adjectives ("fast", "nice", "secure" are not values).
3. Error cases and permissions are included.
4. Out of scope is written down.
5. Two people reading it would agree on pass or fail. (Same test as for eval tasks in part two.)

And the shortcut: if you can't write the test for it, the AC isn't done.

Sources
https://addyo.substack.com/p/agentic-verification

# “Must never” ACs

Examples say what should happen. Rules say what must never happen.

`always equal`

**Cart total = sum of its lines**

`never below`

**A balance never goes negative**

`twice = once`

**Saving twice gives the same result as once**

A **property test** makes up thousands of inputs and tries to break the rule. When it finds one, it shrinks it to the smallest failing case.

# Must never ACs

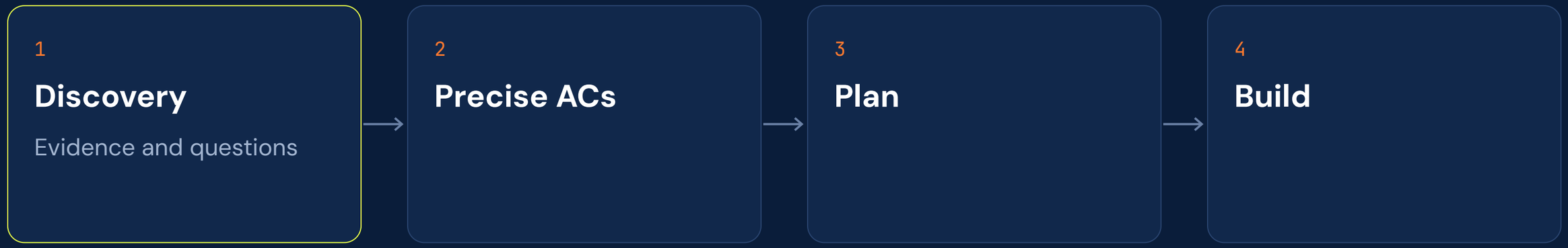
Most ACs only describe the happy path. Every plan should also list a few rules that must always hold: the cart total always equals the sum of its lines, a balance never goes negative, running something twice gives the same result as once.

A property test checks such a rule. You write the rule; the tool generates hundreds or thousands of random inputs (empty lists, huge numbers, odd decimals) and checks the rule on each. When it finds a failing input, it shrinks it to the smallest one, which is easy to read and fix.

Tools: FsCheck or CsCheck for .NET, fast-check for TypeScript, Eris for PHP.

Sources
https://addyo.substack.com/p/agentic-verification

# Planning starts with discovery



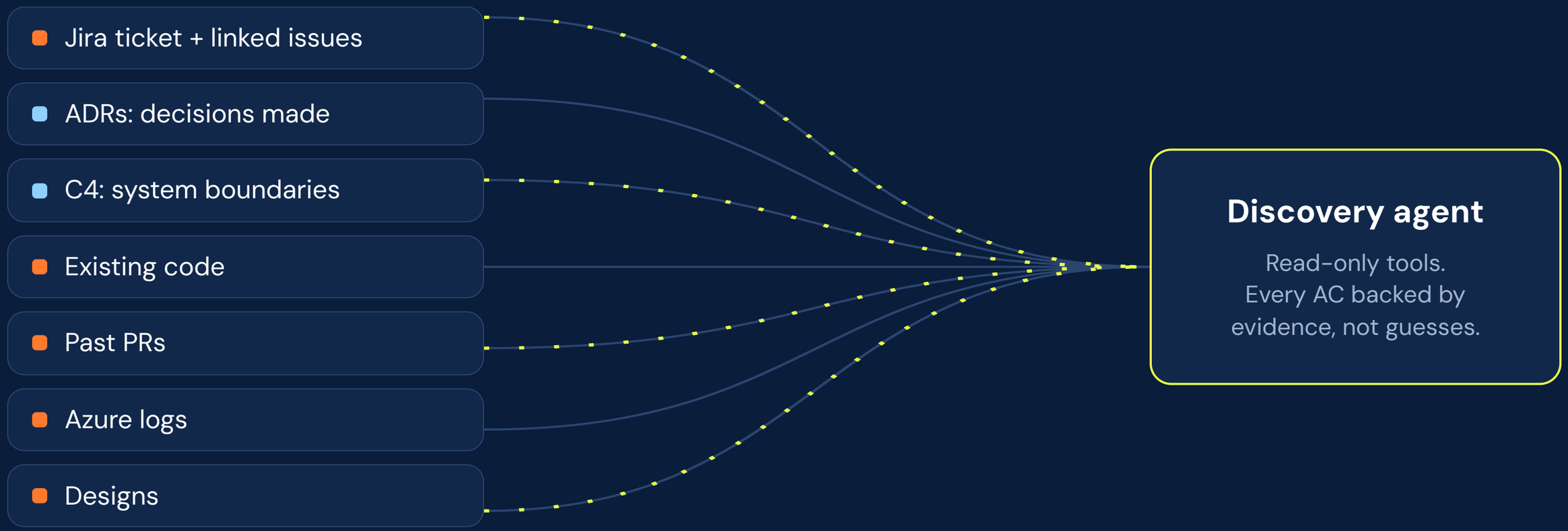

If the ACs aren't crystal clear for the agent, no plan can be made.

# Planning starts with discovery

Before any plan: collect the evidence. If the ACs that together make up the intent aren't crystal clear for the agent, no plan can be made.

The order: discovery → precise ACs → plan → build.

# The discovery agent gathers evidence



# The discovery agent gathers evidence

The discovery agent is a custom agent or a skill that reads every source of evidence:

- the Jira ticket and linked issues
- ADRs: decisions the team already made
- C4 diagrams: system boundaries the solution must respect
- the existing code and past PRs
- Azure logs
- designs

Every AC is backed by evidence, not guesses. It only has read-only tools, so it can't change anything.

Open question: where do Jakamo's tickets and docs live (Jira, Confluence, Azure DevOps)?

# Then it questions you

The ticket says “phone number”. The ADR-012 says we store all contact data in E.164. Which numbers must be accepted?

Finnish only | Any E.164 | Anything with digits

Any E.164.

What happens on a fast double-click on Save? Past PR #482 had duplicate writes here.

Ignore | Limit: 2 saves per second | Disable the button

Limit to 2 per second, the third gets 429.

Every answer is saved as a decision → AC1, AC2

# Then it questions you

After reading the evidence, the agent asks the human, as long as needed. Multiple choice where possible, so answering is fast. It doesn't stop until each AC is precise.

Every answer is written down as a decision, so later you can see why an AC is the way it is.

The ADR number and PR number are made-up examples.

# Example: `/discover JIRA-1234`




**Fetch**
the ticket

**Read**
ADRs, C4, code, PRs

**List gaps**
and conflicts

**Ask**
until precise

**Write**
the ACs file

# Example: /discover JIRA-1234

What the skill does, step by step:

1. Fetch the ticket.
2. Read the ADRs, C4 diagrams, related code and past PRs.
3. List gaps and conflicts between them.
4. Ask the human questions until every AC is precise.
5. Write the ACs file.

If you have your own discovery skill from your project, show it here.

# The output: the ACs file

plans/JIRA-1234/acs.md

```
## AC1 Phone must be E.164
Given a logged-in user, when they save a phone
number that isn't E.164, then the save is blocked
and the field shows "Invalid phone number".
Evidence: ADR-012 · Decision: any E.164 (Mikko)

## AC2 Save rate limit
... the third save within 1 s gets HTTP 429.
Evidence: PR #482 · Decision: 2 per second

## Must never
- A saved phone is never stored in another format

## Out of scope
- SMS verification
```

- Each AC with evidence and decisions
- “Must never” rules
- Out of scope
- This file is the intent

# The output: the ACs file

The output of discovery is one file. Each AC has its evidence link and the decisions behind it. The "must never" rules are listed. Out of scope is listed.

This file is the intent. Everything after this (plan, tests, review) points back to it.

# From ACs to plan

```
$ claude --permission-mode plan
```

- Plan mode: Claude can't edit files until you approve
- Every step in the plan points to the AC it serves

```
1. Add E.164 validator to ProfileRequestValidator   → AC1
2. Show "Invalid phone number" in ProfileForm.tsx  → AC1
3. Add rate limit policy "profile-save"             → AC2
```

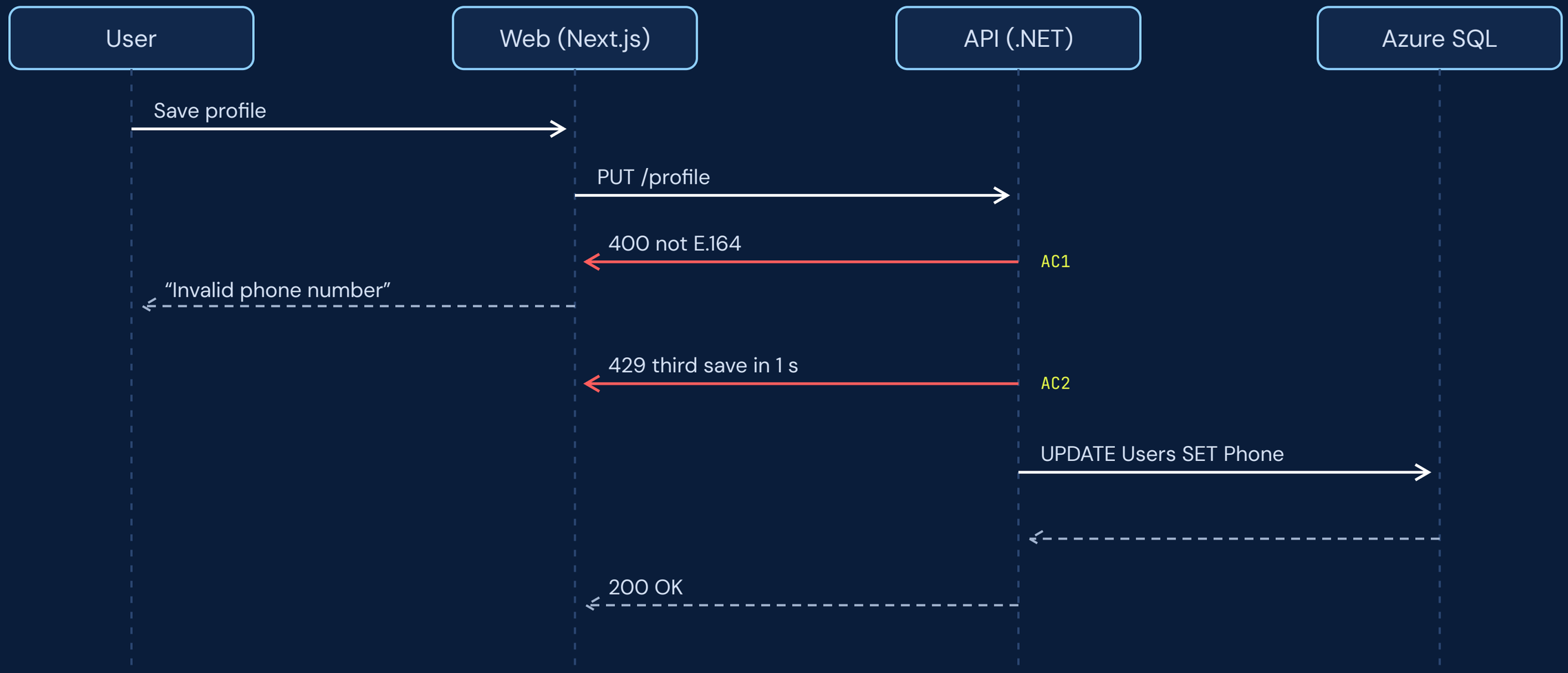
# From ACs to plan

Start Claude in plan mode (claude --permission-mode plan, or switch in the session). In plan mode Claude can read and think but can't edit files until you approve the plan.

Every step in the plan points to the AC it serves. A step with no AC is either missing an AC or not needed.

Sources
https://code.claude.com/docs/en/permission-modes

# Sequence diagram of the solution




As detailed as possible: every actor, call, write and error path. Each step maps to an AC. Gaps show up as missing arrows.

# Sequence diagram of the solution

Part of the plan: a sequence diagram of the solution, as detailed as possible. Every actor, service, API call, database write, queue message and error path. Each step maps to an AC.

The human sees the whole flow before any code exists. Gaps show up as missing arrows: for example, if nothing in the diagram handles a database timeout, ask whether an AC is missing.

For a big feature, draw one diagram per flow so each stays readable.

# The plan names the files

| FILE | CHANGE | WHAT | AC |
|---|---|---|---|
| Api/Profile/ProfileRequestValidator.cs | edit | E.164 rule | AC1 |
| web/profile/ProfileForm.tsx | edit | Error message | AC1 |
| Api/Program.cs | edit | Rate limit policy | AC2 |
| Api.Tests/ProfilePhoneTests.cs | new | Test per AC | AC1, AC2 |
| Api.Tests/PhoneProperties.cs | new | Property test | Must never |
| web/e2e/profile.spec.ts | new | E2E main flow | AC1 |

Files changed + files created. One test per AC.

# The plan names the files

The plan lists every file that will be changed and every file that will be created: one test per AC, a property test per "must never" rule, and an E2E test for the main user flow.

The file names are made-up examples for the stack.

# The file list is a drift guard

- ✓ `ProfileRequestValidator.cs` in the plan
- ✓ `ProfileForm.tsx` in the plan
- ! `Shared/StringUtils.cs` not in the plan → question
- ✗ `PhoneProperties.cs` missing → “must never” rule not covered

Mechanical, so machines check it (part 5).

# The file list is a drift guard

At review, the diff is compared with the plan's file list.

- A changed file that is not in the plan is a question: why did the agent touch it?
- A planned test file that is missing means an AC (or a must-never rule) is not covered.

This check is mechanical, so machines can do it. Part five shows it running in CI.

# Tests come from the intent, not the code

SAME AGENT WRITES BOTH

Feature and test share the same misunderstanding. Both pass. Both are wrong.

OUR WAY

Tests come from the human-approved ACs, before any code exists.

# Tests come from the intent, not the code

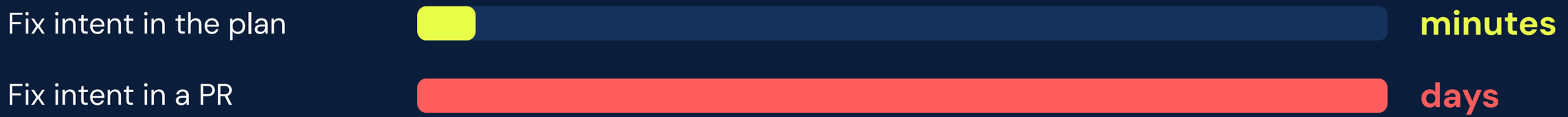
A known problem: if the same agent writes the feature and its test, both can share the same mistake. The test passes, but it tests the wrong thing.

In this flow the tests come from the ACs that a human approved, before any code exists. The test checks the intent, not the code the agent happened to write.

Sources
https://addyo.substack.com/p/agentic-verification

# Humans review intent here

- The ACs
- The sequence diagram
- The plan



# Humans review intent here

Approving the ACs, the sequence diagram and the plan IS the human review. This is where the human decides whether we are building the right thing.

Fixing intent before any code exists takes minutes. Finding the same problem in a PR costs much more: the code, tests and review all have to be redone.

# Build in a fresh session

commit

**plans/JIRA-1234/**

acs.md · sequence.md · plan.md

→

/clear

**New session**

clean context

→

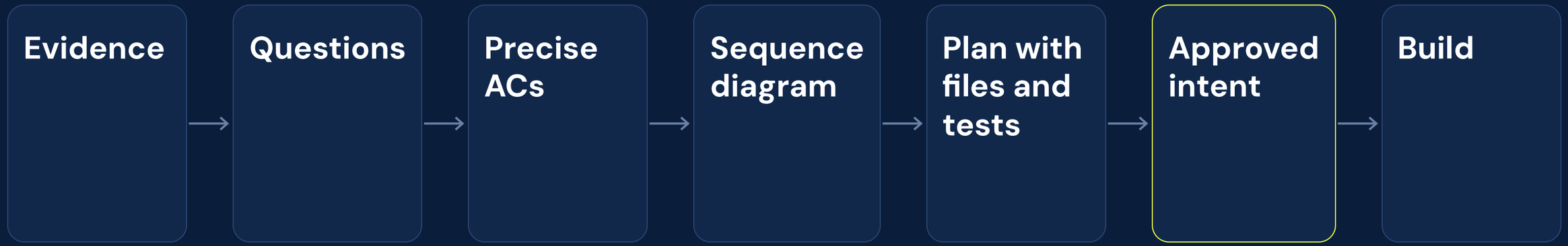
run

**/implement JIRA-1234**

# Build in a fresh session

The approved plan is committed as files. A new session builds from it with a clean context, so the discovery chat doesn't distract it. /implement reads the plan and works through it.

# Recap




Next: how machines check the result.

# Planning recap

The whole planning flow in one line: evidence → questions → precise ACs → sequence diagram → plan with files and tests → approved intent → build.

Next part: how machines check that the result matches the intent.

WUNDERDOG

05

# Validation at scale

# 05 Validation at scale

Part five: validation at scale. The setup is trusted (part two) and the intent is approved (part four). Now the result has to prove it matches both, without a human reading every line.

This is where code review gets solved: machines check that the code is what it should be, humans check the evidence against the intent.

# Where we are

**Setup trusted**
Part 2

**Intent approved**
Part 4

**Code being written**
by the agent

How do we know the result matches the intent, without reading every line?

# Where we are

Recap of where we are: the setup is trusted, the intent is approved, and the agent is writing code. The question for this part: how do we know the result matches the intent, without a human reading every line?

# Two review layers

LAYER 1

## In the implementation loop

Every turn, while the agent works

LAYER 2

## At the PR

Before merge

Most problems should die in layer 1.

# Two review layers

Code review happens in two layers.
Layer 1 runs inside the implementation loop, on every turn, while the agent works. Problems found here are fixed before anyone sees the code.
Layer 2 runs at the PR, before merge.

Most problems should die in layer 1. It is faster and cheaper.

# Static checks while the agent works

.NET

## Analyzers

Nullable on, warnings as errors

PHP

## PHPStan

Highest level you can reach

Next.js

## `tsc` + ESLint

Strict mode

Hooks and the Stop hook run them on every change. The agent fixes them before anyone sees the code.

# Static checks while the agent works

The first layer: static checks that run while the agent works. The hooks from part two run build, analyzers, lint and types on every change, and the Stop hook makes sure they pass before Claude finishes.

Per stack: .NET analyzers with nullable on and warnings as errors; PHPStan on the highest level you can reach; tsc in strict mode and ESLint for Next.js.

These checks are free, fast and never get tired.

# Dead code, complexity, duplication

Agents write more code than needed.

TYPESCRIPT / JAVASCRIPT

**Fallow** finds unused code, copy-paste and complexity hotspots

.NET / PHP

The same checks, written into the review skill

# Dead code, complexity, duplication

Agents tend to add code: new helpers next to existing ones, copy-paste, functions that grow too complex, code nobody calls.

For TypeScript and JavaScript, Fallow finds unused files and exports, duplicated logic (even with renamed variables) and complexity hotspots. For .NET and PHP, put the same checks into the review skill, or use the quality platform on slide "Code quality platforms".

Sources
https://blog.logrocket.com/clean-ai-code-fallow/

# AC validation in the loop

CHECK 1

## The tests

- One test per AC
- Property tests for “must never”
- E2E for the main flow

CHECK 2

## The implementation

- Checked against each AC
- Nothing missing, nothing extra

AC1 ✓ AC2 ✓ Must never ✓ → only then can the agent finish

# AC validation in the loop

Every AC is checked twice in the loop:

1. The tests: one test per AC, property tests for the must-never rules, and an E2E test for the main flow.
2. The implementation is checked against each AC: nothing missing, nothing extra.

The agent can't finish until every AC is green (the Stop hook from part two).

# Don't let the agent game the tests

**Test fails**

AC2: expected 429, got 200

→

**Agent tries to edit the test**

`Edit ProfilePhoneTests.cs`

→

**Hook blocks it**

Approved test files are locked during /implement. Fix the code.

# Don't let the agent game the tests

When a test fails, the easiest way out for the agent is to change the test. A PreToolUse hook blocks edits to the approved test files during /implement. The agent has to fix the code, not the test.

This connects part two (hooks) and part four (tests come from the intent).

# Fast and deterministic

**Seconds**
Core checks, one command

**Same result**
Fixed seeds, frozen clocks, isolated state

**Short output**
Only what failed

**No flaky tests**
Remove them right away

A flaky test teaches the agent that retrying beats fixing.

# Fast and deterministic

The agent learns from the loop. Make the loop fast and deterministic:

- Core checks run in seconds with one command.
- The same input gives the same result: fixed random seeds, frozen clocks, isolated state.
- The output shows only what failed. Noise costs attention on every turn.
- Flaky tests teach the agent that retrying works better than fixing. Remove them from the main suite right away. A flaky test is worse than none.

Sources

https://addyo.substack.com/p/agentic-verification

# Docs travel with the code

**PR #1234 · Phone number on profile**

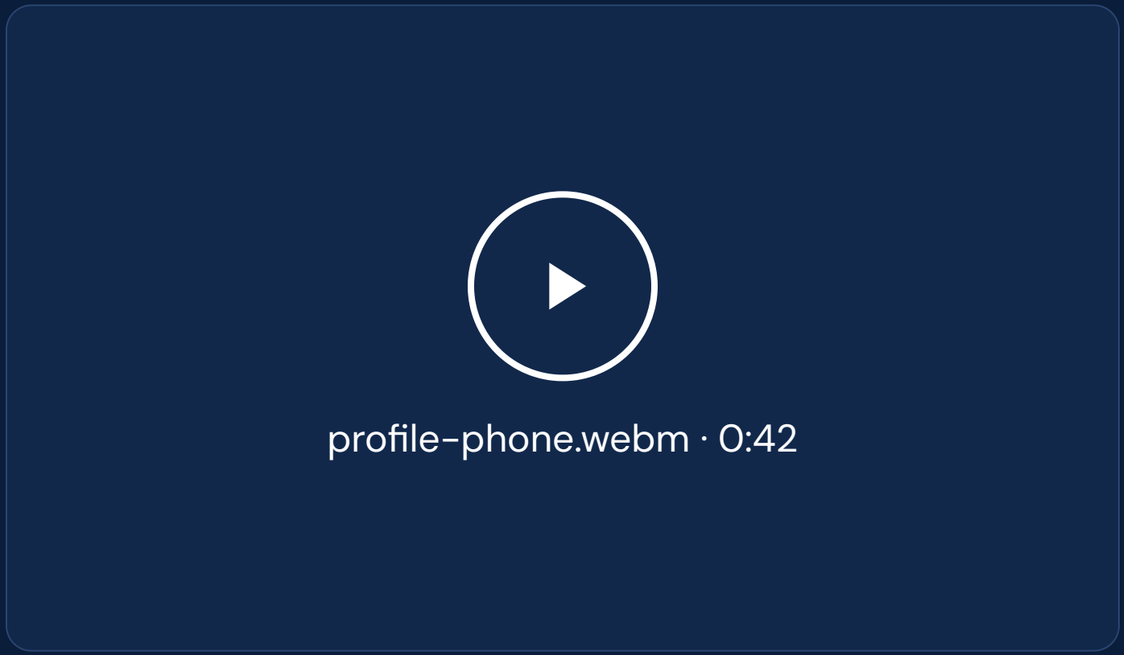
- Documentation updated
- ADR-012 updated
- Changelog entry — missing → check fails

# Docs travel with the code

The PR must include the documentation updates, ADR updates and a changelog entry, either in the change or in the PR body. If one is missing, the check fails.

This is easy to check mechanically, and it means docs never fall behind the code.

# Show it running




- A recording of the implementation running
- Video, E2E trace or screenshots of the user flow
- Attached to the PR

# Show it running

Every PR includes a recording of the implementation running: a video, a Playwright E2E trace, or screenshots of the user flow. The reviewer sees the feature working without checking out the branch.

# Sequence diagram, round two

FROM THE PLAN

```
User → Web: Save profile
Web → API: PUT /profile
API → Web: 400 not E.164   AC1
API → Web: 429 rate limit  AC2
API → SQL: UPDATE Users
```

DRAWN FROM THE CODE

```
User → Web: Save profile
Web → API: PUT /profile
API → Web: 400 not E.164   AC1
API → Web: 429 ... missing
API → SQL: UPDATE Users
API → Cache: SET profile  (new)
```

Every difference is drift.

# Sequence diagram, round two

A review agent draws the sequence diagram from the actual code and compares it with the one from the plan. Every difference is drift: here the rate limit response is missing (AC2 not implemented) and there is a new cache write nobody planned.

This catches drift in behavior, not only in files.

# The file drift guard

```
$ drift-guard --plan plans/JIRA-1234/plan.md
  ✓ 5 of 6 planned files changed
  ! Shared/StringUtils.cs changed, not in the plan
  ✗ Api.Tests/PhoneProperties.cs missing (must-never rule)
```

Posted as a PR comment. Machines ask the question, not the reviewer.

# The file drift guard

The drift guard from part four, running in CI. A script compares the PR's changed files with the plan's file list. A file outside the plan is flagged. A missing test file means an AC is not covered. The result is posted as a PR comment.

The command name is an example; it's a short script you can write for your CI.

# The PR lists the evidence

- AC1 → ProfilePhoneTests.InvalidPhone → passed
- AC2 → ProfilePhoneTests.RateLimit → passed
- Must never → PhoneProperties (1,000 inputs) → passed
- Docs, ADR-012, changelog
- Recording · diagram comparison · drift report

The human reads evidence, not the diff.

# The PR lists the evidence

The PR description lists the evidence: each AC with its test and result, the must-never property tests, the docs, ADR and changelog, the recording, the diagram comparison and the drift report.

The human reads the evidence against the approved intent. That takes minutes, not hours.

Sources
https://addyo.substack.com/p/agentic-verification

# Bonus: grade the PR

## Small

Small change, no business logic

Auto-merge

## Medium

Bigger change, no business logic

Machine review

## Large

Many files or packages, touches business logic

Needs a human

## Critical

Core business logic, migrations, database layer

Human gate, mandatory

# Bonus: grade the PR

Grade every PR so people only look where it matters:

- Small: a small change with no business logic. Auto-merge.
- Medium: a bigger change, still no business logic. Machines review it.
- Large: many files or packages, touches business logic. Needs a human.
- Critical: core business logic, migrations, the database layer. A human gate is mandatory.

Open question: should medium auto-merge, or get a quick human look?

# Review providers

### CodeRabbit

AI review comments on every PR

### Greptile

Reviews with the whole codebase as context

### Claude in CI

Claude Code in your pipeline, with your own review skill

- Comment on every PR and catch what rules miss
- Helpful, but they are models, not guarantees

# Review providers

AI review tools comment on every PR: CodeRabbit, Greptile, or Claude running in CI. They catch things no rule covers.

They are models, so they are not guarantees. Treat their comments as input for the human, not as a gate.

# Code quality platforms

SonarCloud: one quality gate for all three stacks

**Coverage**

**Duplication**

**Complexity**

**Security issues**

.NET PHP TypeScript

# Code quality platforms

A code quality platform such as SonarCloud gives one quality gate for coverage, duplication, complexity and security issues across .NET, PHP and TypeScript. It covers what Fallow doesn't for .NET and PHP.

# Evals in CI: know why it broke

WITHOUT EVALS

Build red. Something changed.

WITH EVALS

```
uses-migration-skill 1.00 → 0.33
after: CLAUDE.md edit in PR #1301
```

When the setup or the model changes, evals show which behavior changed.

# Evals in CI: know why it broke

The evals from part two run in CI. When someone changes the setup, or a new model comes out, the evals show which behavior changed, not just that the build went red.

The numbers are example data.

Sources
https://code.claude.com/docs/en/plugin-evals

# The hidden value of review comments

every week

**A scheduled workflow triages the comments**

→

shared place

**Findings posted for the team**

→

team

**The team discusses them**

→

fix once

**Common ones become rules, hooks or tests**

The same comment never has to be written twice.

# The hidden value of review comments

Review comments are the best record of what the team cares about. A scheduled workflow reads the review comments once a week, groups them and posts the findings to a shared place. The team discusses them, and the common ones become rules, hooks or tests.

This is how the setup gets better every week.

Open question: where should the findings go at Jakamo (Teams, Confluence)?

# What humans still do

### Approve the intent

ACs, sequence diagram, plan

### Read the evidence

Not every line of the diff

### Own large and critical PRs

Business logic, migrations, database

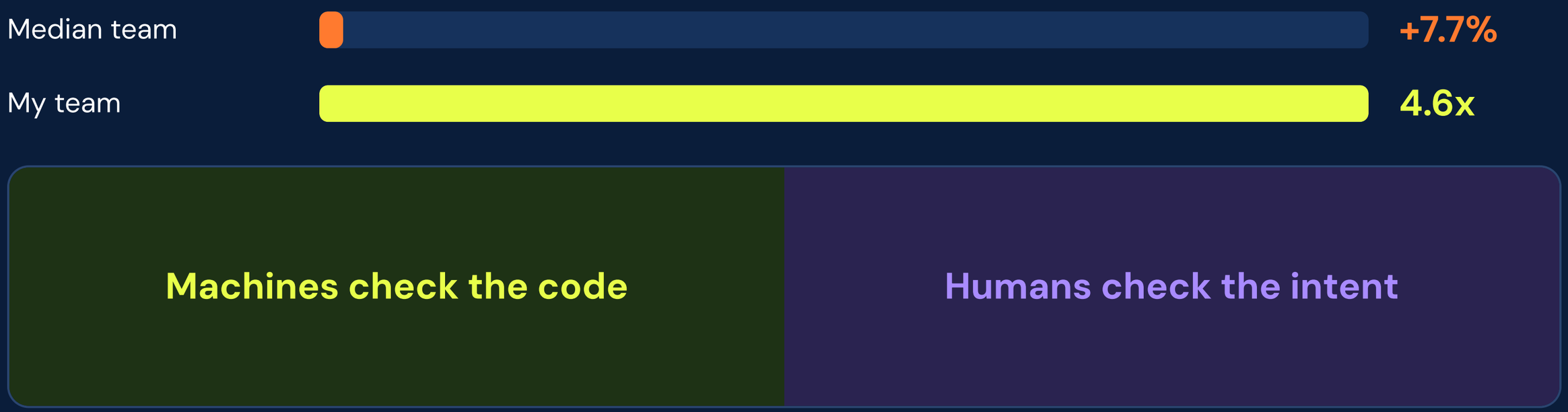
### Be accountable

A person is responsible for what ships

# What humans still do

Humans don't disappear. They approve the intent, read the evidence, own the large and critical PRs, and stay accountable for what ships.

# Code review is solved




Next: the hackathon.

# Code review is solved

Back to part one: the median team got about 7.7%. Your team got 4.6x. The difference is the review bottleneck: machines check the code, humans check the intent.

Then hand over to the hackathon: pick one technique from today and build it for your own team.

WUNDERDOG

# Hackathon
# Build one check for your team

Thank you · questions

# Hackathon

Thank the room. Take questions. Then start the optional hackathon: each team picks one pain point and builds one check, hook, skill or plan template for it.